

HACKER ATTACK

PROTEÇÕES SOBRE O CÓDIGO



4

LISTA DE FIGURAS

Figura 1– Code Hardening	6
Figura 2 – Exemplo de ofuscação de código.....	7
Figura 3 – Criptografia.....	10
Figura 4 – Execução do LAB de ofuscação – 1.....	15
Figura 5 – Execução do LAB de ofuscação – 2.....	15
Figura 6 – Execução do LAB de ofuscação – 3.....	15



LISTA DE CÓDIGOS-FONTE

Código-fonte 1 – Script em Python.....	8
Código-fonte 2 – Script em Python ofuscado	8
Código-fonte 3 – Script original em Shell	9
Código-fonte 4 – Sequência de ofuscação em Shell.....	9
Código-fonte 5 – Desofuscação em Shell	9
Código-fonte 6 – Script para criptografia em Python – 1.....	11
Código-fonte 7 – Script para criptografia em Python – 2.....	11
Código-fonte 8 – LAB de ofuscação – 1.....	13
Código-fonte 9 – LAB de ofuscação – 2.....	14

SUMÁRIO

HARDENING SOBRE O CÓDIGO	5
1 INTRODUÇÃO	5
2 HARDENING	5
3 OFUSCAÇÃO: QUE TAL ENGANAR O INIMIGO?	6
3.1 Implementando ofuscação em Python	8
3.2 Implementando ofuscação em Shell Script	8
4 UNBREAKABLE CODE: USANDO CRIPTOGRAFIA EM SEUS CÓDIGOS.....	9
4.1 Usando o Python para criptografar códigos	10
5 LAB – HARDENING DE CÓDIGO COM PYTHON	11
6 CONCLUSÃO.....	15
REFERÊNCIA	17

HARDENING SOBRE O CÓDIGO

1 INTRODUÇÃO

Avançando na esfera do conhecimento que tange proteger e blindar o código de uma aplicação, veremos, neste capítulo, técnicas de proteção sobre aplicações e, posteriormente, avaliaremos boas práticas para o desenvolvimento seguro. Não adianta contarmos com uma equipe altamente treinada, o melhor hardware, equipamentos para segurança, banco de dados seguro, se a aplicação, que é responsável por manipular os dados, contiver falhas de segurança. Por isso, devemos continuar com muita atenção neste ponto que será o grande diferencial do seu trabalho dentro da “FIAP COIN”.

2 HARDENING

Quando se desenvolve um código sob a cultura do software livre, é normal que compartilhemos nossos *sources* para que outros desenvolvedores possam melhorá-lo. Porém, quando falamos de desenvolvimento para *Cybersecurity*, nem sempre nosso código pode ser visto por terceiros.

Imagine que a aplicação do seu cliente “FIAP COIN” esteja rodando em um *web server*, por exemplo. Seguindo o conceito de Defesa em Profundidade, não só a aplicação e o servidor precisam de cuidados contra atacantes, mas também seu código rodando naquele ambiente. Se em um cenário de ataque (ou mesmo de um Pentest), um atacante conseguir acesso ao Document Root do seu *web server*, estando seu código “hardenizado”, você teria uma camada a mais de proteção em seu ambiente.

Neste capítulo, iremos falar sobre algumas técnicas de proteção (ou Hardening) de códigos, e como elas são úteis e simples de implementar em nossos cenários.



Figura 1– Code Hardening
Fonte: Adaptado por Fiap (2019)

3 OFUSCAÇÃO: QUE TAL ENGANAR O INIMIGO?

Ocultação de código, ou ofuscação, é o ato de obscurecer deliberadamente o código-fonte, dificultando a compreensão dos humanos e tornando-o inútil para atacantes que possam ter segundas intenções, como, por exemplo, a engenharia reversa de software.

Essencialmente, a ofuscação altera completamente o Código-Fonte; no entanto, permanece funcionalmente equivalente ao código original.

Ofuscação NÃO é o mesmo que criptografia. O objetivo da criptografia é transformar dados a fim de mantê-los em segredo dos outros. O objetivo da ofuscação é dificultar o entendimento dos dados pelos humanos. O código criptografado sempre precisa ser descriptografado antes da execução, enquanto a ofuscação não exige que o código sofra a “desofuscação” para executá-lo.

E como funciona a ofuscação?

A ofuscação de código consiste em várias técnicas diferentes, cada uma construindo uma sobre a outra, tornando o código ininteligível. A seguir, são apresentadas algumas das técnicas de ofuscação utilizadas:

1. **Renomear:** renomear basicamente modifica os nomes de variáveis e métodos, dificultando o código para o ser humano entender. No entanto, ele ainda mantém o comportamento de execução do programa. Essa técnica básica é mais comumente usada para ofuscadores Android, Java e IOS.
2. **Criptografia de string:** embora a técnica de renomeação simplesmente altere os nomes de variáveis e métodos, a criptografia de string pretende criptografar todas as strings que são claramente legíveis. (Você deve observar que, com essa técnica, descriptografar sequências de caracteres no tempo de execução pode resultar em uma diminuição de performance em tempo de execução).
3. **Inserção de código fictício:** isso tem a ver com a inserção de código no executável. Essa técnica dificulta a análise do código de engenharia reversa. No entanto, as inserções não afetam a lógica e/ou a execução do programa.

Antes da Ofuscação:

```
1. função sayHello ( ) {  
2.   console.log ( "Olé Mundo" ) ;  
3. }  
4.  
5. sayHello ( ) ;
```

Após Ofuscação:

```
1. var _0x12b3 = [ "\ x48 \ x65 \ x6C \ x6C \ x6F \ x20 \ x57 \ x6F \ x72 \ x6C \ x64" , "\ x6C \ x6F \ x67" ] ;  
2. função sayHello ( ) { console [ _0x12b3 [ 1 ] ] ( _0x12b3 [ 0 ] )  
} sayHello ( )
```

Figura 2 – Exemplo de ofuscação de código
Fonte: Google Imagens (2019)

As principais linguagens de programação possuem suporte a técnicas de ofuscação, como é o caso do C, Java, PHP, Python, etc., porém, nos próximos tópicos, iremos abordar as duas linguagens mais utilizadas em ambientes de segurança: Python e Shell Script.

3.1 Implementando ofuscação em Python

Por se tratar de uma linguagem bem modular e flexível, existem alguns módulos prontos em Python que nos auxiliam no processo de ofuscação, como o **base64**, por exemplo.

Imagine um script em Python bem simples como este:

```
print "Hello World!"
```

Código-fonte 1 – Script em Python
Fonte: Elaborado pelo autor (2019)

Se formos implementar a ofuscação dessa linha de código usando o módulo base64 através do terminal interativo do Python, ficariam assim:

```
>>> import base64

>>> mycode = "print 'Hello World!'"

>>> secret = base64.b64encode(mycode)

>>> secret

'cHJpbnQgJ2h1bGxvIFdvcmxkICEn'

>>> mydecode = base64.b64decode(secret)

>>> eval(compile(mydecode,'<string>','exec'))

Hello World!
```

Código-fonte 2 – Script em Python ofuscado
Fonte: Elaborado pelo autor (2019)

3.2 Implementando ofuscação em Shell Script

Para scripts em Bash (Shell Script), o processo é o mesmo e, inclusive, podemos utilizar o **comando base64** (nativo do GNU/Linux) para realizar o processo de ofuscação.

Usemos como o exemplo este simples script em shell abaixo, no qual chamaremos de **original**:


```
#!/bin/sh

echo "foo"
```

Código-fonte 3 – Script original em Shell
Fonte: Elaborado pelo autor (2019)

Abaixo, temos um exemplo de uma sequência de comandos que poderíamos utilizar para ofuscar nosso script inicial, gerando um novo (ofuscado) chamado de obsf:

```
$ echo "echo $(base64 origin)" > obsf

$ cat obsf

echo lyEvYmluL3NoCmVjaG8gImZvb28iCg==

$ chmod +x obsf
```

Código-fonte 4 – Sequência de ofuscação em Shell
Fonte: Elaborado pelo autor (2019)

Logo, para executar nosso shell script ofuscado, usamos:

```
$ sh obsf | base64 -d | sh

foo
```

Código-fonte 5 – Desofuscação em Shell
Fonte: Elaborado pelo autor (2019)

4 UNBREAKABLE CODE: USANDO CRIPTOGRAFIA EM SEUS CÓDIGOS

A Criptografia é, basicamente, o processo de converter uma mensagem de texto em texto cifrado, que pode ser decodificado novamente na mensagem original, sendo uma importante aliada na iniciativa de manter seus dados seguros.

Atualmente existem muitos algoritmos de criptografia diferentes; alguns dos mais usados são RSA, Triple DES, Blowfish e AES.



Figura 3 – Criptografia
Fonte: Google Imagens (2019)

É inegável que, do ponto de vista de segurança, a criptografia seja muito útil, porém, na perspectiva da entregabilidade da sua aplicação, ela diminui, consideravelmente, a performance, visto que, para executar seu código, é necessária a descryptografia em tempo de execução.

4.1 Usando o Python para criptografar códigos

O código abaixo utiliza o módulo Cipher (PyCrypto) e suas funções Crypto e o algoritmo AES para implementar uma camada de criptografia em uma string. Obviamente que, neste exemplo, optamos por utilizar uma string por ser mais didático, porém você pode utilizar como entrada deste script de criptografia o seu código-fonte, por exemplo.

```
# AES pycrypto package

from Crypto.Cipher import AES

# key has to be 16, 24 or 32 bytes long

encrypt_AES = AES.new('secret-key-12345', AES.MODE_CBC, 'This is an
IV-12')

# Fill with spaces the user until 32 characters

message = "This is the secret message    "

ciphertext = encrypt_AES.encrypt(message)

print("Cipher text: ", ciphertext)

# key must be identical
```

```
decrypt_AES = AES.new('secret-key-12345', AES.MODE_CBC, 'This is an
IV-12')

message_decrypted = decrypt_AES.decrypt(ciphertext)

print("Decrypted text: ", message_decrypted.strip())
```

Código-fonte 6 – Script para criptografia em Python – 1
Fonte: Elaborado pelo autor (2019)

A saída deste script seria esta:

```
('Cipher text: ',
'\xf2\xda\x92:\xc0\xb8\xd8PX\xc1\x07\xc2\xad"\xe4\x12\x16\x1e)(\xf4\xae\xdeW\xaf
_\x9d\xbd\xf4\xc3\x87\xc4')('Decrypted text: ', 'This is the secret message')
```

Código-fonte 7 – Script para criptografia em Python – 2
Fonte: Elaborado pelo autor (2019)

Com este script de criptografia de códigos em Python, você será capaz de criptografar códigos escritos em outras linguagens. Para isso, basta usá-los como entrada para as funções criptográficas aqui apresentadas.

5 LAB – HARDENING DE CÓDIGO COM PYTHON

Neste LAB, iremos utilizar a técnica de ofuscação para transformarmos uma simples string em um shellcode.

Utilizando sua VM, do Kali Linux, abra seu terminal e, por meio do seu editor de texto preferido, copie e cole o código abaixo:

```
source=""

print 'Hello World!'

'''

def obfuscate(source):

    # source = source.replace(' ', '\x09')

    return "".join('%02x' % ord(c) for c in source)

code = obfuscate(source)
```

```
print code

print

temp = ""\x' + '\x'.join(code[i:i+2] for i in range(0, len(code), 2)) + ""

print 'temp:', temp, 'x'

'''

# sample

def MakeSC():

    c = raw_input(" Encode: ")

    sc = ""\x" + ""\x".join("{0:x}".format(ord(c)) for c in c)

    print "\n shellcode =( " + sc + " ); exec(shellcode)"

test_code =
'\x0a\x70\x72\x69\x6e\x74\x20\x27\x48\x65\x6c\x6c\x6f\x20\x57\x6f\x72\x6c\x64\x2
1\x27\x0a'

print

print 'test:', test_code

print

print 'executing:'

exec(test_code)

header =
'\x66\x72\x6f\x6d\x20\x62\x61\x73\x65\x36\x34\x20\x69\x6d\x70\x6f\x72\x74\x20\x
62\x36\x34\x64\x65\x63\x6f\x64\x65\x20\x61\x73\x20\x5f'

exec(header)

code = 'CnByaW50lCdlZWxsbyBxb3JsZCEnCg=='

exec_(_('CnByaW50lCdlZWxsbyBxb3JsZCEnCg=='))

#from base64 import b64decode as _

#def __(_):exec(_)
```

```
#print

#enc = base64.b64encode(source)

#print enc

#exec(base64.b64decode(code))

'''
```

Código-fonte 8 – LAB de ofuscação – 1
Fonte: Elaborado pelo autor (2019)

Salve seu script como ***ofuscador.py***. Substituindo a string Hello World! na variável **source**, insira qualquer conteúdo que deseje para transformar em shellcode, conforme abaixo:

```
source=""

print 'Defesa Cibernetica eh o melhor curso da FIAP'

'''

def obfuscate(source):

#   source = source.replace(' ', '\x09')

    return "".join('%02x' % ord(c) for c in source)

code = obfuscate(source)

print code

print

temp = ""'\x' + '\x'.join(code[i:i+2] for i in range(0, len(code), 2)) + ""

print 'temp:', temp, 'x'

'''

# sample

def MakeSC():

    c = raw_input(" Encode: ")

    sc = ""'\x' + ""'\x'.join("{0:x}".format(ord(c)) for c in c)
```

```
print "\n shellcode =('" + sc + "'); exec(shellcode)"

test_code =
'\x0a\x70\x72\x69\x6e\x74\x20\x27\x48\x65\x6c\x6c\x6f\x20\x57\x6f\x72\x6c\x64\x2
1\x27\x0a'

print

print 'test:', test_code

print

print 'executing:'

exec(test_code)

header =
'\x66\x72\x6f\x6d\x20\x62\x61\x73\x65\x36\x34\x20\x69\x6d\x70\x6f\x72\x74\x20\x
62\x36\x34\x64\x65\x63\x6f\x64\x65\x20\x61\x73\x20\x5f'

exec(header)

code = 'CnByaW50lCdlZWxsbyBXb3JsZCEnCg=='
exec_ ('CnByaW50lCdlZWxsbyBXb3JsZCEnCg==')

#from base64 import b64decode as _
#def __(_):exec(_)
#print
#enc = base64.b64encode(source)
#print enc
#exec(base64.b64decode(code))

'''
```

Código-fonte 9 – LAB de ofuscação – 2

Fonte: Elaborado pelo autor (2019)

Agora basta executá-lo para gerar sua string em shellcode, conforme abaixo:

```

v1n1@MrR00b0t: ~
v1n1@MrR00b0t:~$ python ofuscar.py
0a7072696e7420274465666573612043696265726e6574696361206568206f206d656c686f7220637572736f2064612046494150270a

temp: "\x0a\x70\x72\x69\x6e\x74\x20\x27\x44\x65\x66\x65\x73\x61\x20\x43\x69\x62\x65\x72\x6e\x65\x74\x69\x63\x61\x20\x65\x68\x20\x6f\x20\x6d\x65\x6c\x68\x6f\x72\x20\x63\x75\x72\x73\x6f\x20\x64\x61\x20\x46\x49\x41\x50\x27\x0a" x
v1n1@MrR00b0t:~$

```

Figura 4 – Execução do LAB de ofuscação – 1
Fonte: Elaborado pelo autor (2019)

Você deve ter reparado que no script de ofuscação que estamos utilizando existem outras funções, como, por exemplo, uma que desofusca o shellcode, transformando-o em string novamente (função MakeSC). Para usá-la, basta descomentá-la no script e alterar a variável **test_code**, inserindo o seu shellcode gerado no passo anterior.

```

source='''
print 'Defesa Cibernetica eh o melhor curso da FIAP'

def obfuscate(source):
    source = source.replace(' ', '\x09')
    return ''.join('%02x' % ord(c) for c in source)

code = obfuscate(source)
print code

print
temp = '\\x' + '\\x'.join(code[i:i+2] for i in range(0, len(code), 2)) + ''
print 'temp:', temp, 'x'

# sample

def MakeSC():
    c = raw_input(" Encode: ")
    sc = '\\x' + '\\x'.join('%02x' % ord(c) for c in c)
    print "\n shellcode =" + sc + "; exec(shellcode)"

test_code = '\x0a\x70\x72\x69\x6e\x74\x20\x27\x44\x65\x66\x65\x73\x61\x20\x43\x69\x62\x65\x72\x6e\x65\x74\x69\x63\x61\x20\x65\x68\x20\x6f\x20\x6d\x65\x6c\x68\x6f\x72\x20\x63\x75\x72\x73\x6f\x20\x64\x61\x20\x46\x49\x41\x50\x27\x0a'
#test_code = '\x0a\x70\x72\x69\x6e\x74\x20\x27\x44\x65\x66\x65\x73\x6f\x20\x64\x61\x20\x46\x49\x41\x50\x27\x0a'
print
print 'test:', test_code

"ofuscar.py" 46L, 1311C

```

Figura 5 – Execução do LAB de ofuscação – 2
Fonte: Elaborado pelo autor (2019)

```

v1n1@MrR00b0t: ~
v1n1@MrR00b0t:~$ python ofuscar.py
0a7072696e7420274465666573612043696265726e6574696361206568206f206d656c686f7220637572736f2064612046494150270a

temp: "\x0a\x70\x72\x69\x6e\x74\x20\x27\x44\x65\x66\x65\x73\x61\x20\x43\x69\x62\x65\x72\x6e\x65\x74\x69\x63\x61\x20\x65\x68\x20\x6f\x20\x6d\x65\x6c\x68\x6f\x72\x20\x63\x75\x72\x73\x6f\x20\x64\x61\x20\x46\x49\x41\x50\x27\x0a" x

test: '\x0a\x70\x72\x69\x6e\x74\x20\x27\x44\x65\x66\x65\x73\x6f\x20\x64\x61\x20\x46\x49\x41\x50\x27\x0a'
print
print 'Defesa Cibernetica eh o melhor curso da FIAP'

executing:
Defesa Cibernetica eh o melhor curso da FIAP
v1n1@MrR00b0t:~$

```

Figura 6 – Execução do LAB de ofuscação – 3
Fonte: Elaborado pelo autor (2019)

6 CONCLUSÃO

Vimos, neste capítulo, como é simples adicionar camadas de proteção aos nossos códigos com o objetivo de dificultar a leitura por um atacante. Estas técnicas são úteis como forma de boas práticas de desenvolvimento seguro,

porém devem ser muito bem planejadas. Porque, se implementadas de forma errada, podem interferir na performance da sua aplicação.

Por isso, é fundamental contar com um desenvolvimento seguro, em que os códigos gerados para o desenvolvimento do sistema já estejam estruturados de maneira segura. Agora que já vimos técnicas para identificar vulnerabilidades e ampliar segurança, chegou o momento de entendermos algumas boas práticas de segurança que os programadores podem adotar.

EXEMPLO

REFERÊNCIA

BEHERA, C. K.; BHASKARI, D. Lalitha. Different obfuscation techniques for code protection. **Procedia Computer Science**, v. 70, p. 757-763, 2015. Disponível em: <<https://www.sciencedirect.com/science/article/pii/S1877050915032780>>. Acesso em: 11 dez. 2020.

EXEMPLO